

Introduction

Large language models (LLMs) have demonstrated impressive code generation capabilities, yet verifying the correctness of AI-generated code remains a fundamental challenge. The dominant approach uses a separate “judge” model—a discriminator, a reward model, or another LLM—to evaluate code quality. This introduces a circular dependency: the judge itself is fallible, biased, and opaque.

We propose a fundamentally different architecture. Instead of relying on an AI judge to evaluate AI outputs, we distribute the computation across multiple redundant nodes and use *cryptographic hash consensus* to verify that all nodes produced equivalent outputs. The key enablers are:

1. **Deterministic inference.** A model with fixed seed, fixed weights, and no stochastic sampling (temperature=0) is a deterministic function. Identical input yields identical token-by-token output.
2. **AST normalization.** Raw code strings differ in variable names, formatting, and comments. AST normalization strips these superficial differences, revealing the algorithmic structure.
3. **Hash commitment.** The normalized code is hashed via SHA-256. Any node deviating from the consensus hash is flagged as faulty or malicious.

This approach embodies the principle we call “Don’t hire a judge; deploy more players.” Instead of a centralized evaluator that can be spoofed or biased, the system relies on identical computation in replicated environments. Divergence is detectable; convergence is verifiable.

Paper organization.

Section 3 describes the ClawNet protocol in detail. Section 4 presents experimental validation across five code styles, cross-machine execution, and AST normalizer accuracy. Section 5 discusses applications including video generation temporal consistency. Section 6 covers limitations and future work.

Related Work

Code verification.

Existing code verification for LLMs falls into two categories: **execution-based** (running unit tests) and **model-based** (using a second LLM as judge). Execution-based approaches require oracle test cases; model-based approaches are vulnerable to judge bias and position bias.

Byzantine fault tolerance.

Redundant execution across multiple nodes is a well-studied technique in distributed systems. ClawNet applies this to neural inference, where the “state machine” is the model’s forward pass and a fixed random seed.

AST analysis.

Abstract Syntax Tree comparison is standard in plagiarism detection and code clone detection. We repurpose it for cross-node consistency verification.

Method

Protocol Overview

ClawNet operates as a three-phase protocol:

1. **Submission.** A code generation task T (e.g., “write a Python function that filters prime numbers”) is broadcast to N redundant computation nodes.
2. **Computation.** Each node n_i runs the Mother Model with the same seed s , temperature $t=0$ (greedy decoding), and the same prompt p .
3. **Verification.** Each node computes: $h_i = \text{SHA-256}(\text{AST-Normalize}(\text{decode}(\text{model}(p, s))))$ If $i, j, \dots, h_i = h_j$, consensus is reached.

Mother Model

The Mother Model is a compact Transformer decoder trained specifically for deterministic code generation. Key design decisions:

- **Small footprint.** We use a 185M-parameter model (INT4 quantized: ~488 MB), designed to run on consumer GPUs such as the NVIDIA GTX 1060 with 4 GB VRAM.
- **Deterministic runtime.** All dropout is disabled during inference. The random seed is fixed per task. Activation functions (GELU, ReLU) are deterministic in PyTorch.
- **Multi-language output.** The model is trained to emit code in Go, Python, and TypeScript—three languages commonly used in DevOps and cloud-native environments.

AST Normalization

Raw code strings from different model runs vary in:

- Variable names (`n`, `num`, `number`)
- Whitespace, indentation, comments
- Docstrings and redundant annotations

The AST normalizer operates at the Python `ast.NodeTransformer` level (for Python code) and regex-based structural normalization (for Go and TypeScript). It performs:

1. **Variable anonymization:** All user-defined identifiers are replaced with canonical names ($v_0, \dots, v_1, \dots$).
2. **Docstring removal:** Module- and function-level string constants are stripped.
3. **Ternary expansion:** Ternary expressions (a if c else b) are expanded to `if/else` blocks.
4. **Early-return unification:** The pattern `if cond: return a; return b` is rewritten to `if cond: return a`
`else: return b`.

Execution Result Consensus

As a secondary verification layer, ClawNet can optionally execute all candidate code snippets on a common set of test inputs and compare the outputs via hash:

$$h_{\text{exec}} = \text{SHA-256}(\text{serialize}(\{\text{input}_k, \text{output}_k\}))$$

This layer is algorithm-agnostic: syntactically different implementations (Inlined loops vs. helper functions vs. list comprehensions) that compute the same function yield identical h_{exec} . This serves as a cross-model sanity check when multiple different models are available, but is not required for the core single-model consensus.

Experiments

Same-Machine Determinism

We trained a 3M-parameter Transformer decoder on CPU with a synthetic code dataset (8 samples, 30 epochs, final loss 0.0897). Inference was run twice with identical seed ($s=42$) and temperature ($t=1.0$).

Identical seed yields identical output and identical hash. The output is structured code tokens (the tiny model produces plausible but syntactically incomplete Go code, which is expected for a 3M-parameter model trained on 8 samples).

Run	Seed	Output (first 40 chars)	SHA-256
1	42	<code>func add(a in:\n b ret in...</code>	<code>eb6ceca3...</code>
2	42	<code>func add(a in:\n b ret in...</code>	<code>eb6ceca3...</code>

The outputs were byte-for-byte identical. This demonstrates that a PyTorch-based Transformer with deterministic seed control produces *perfectly reproducible* inference.

AST Normalizer Cross-Language Validation

The AutoClaw normalizer was tested on 9 scenarios spanning three languages:

- **Python:** 3 scenarios – a simple filter function, a loop-based equivalent, and a function with intentional variable renaming. All normalized to identical hashes.
- **Go:** 3 scenarios – standard `main()` with print statements, a helper-function decomposition, and a single-function version. Variable renaming normalized.

- **TypeScript:** 3 scenarios “ arrow function, named function, and inline return. Comments and formatting stripped.

All 9 test cases passed.

Execution Consensus Across Code Styles

Five different Python implementations of the same prime-number filter were tested on 7 input cases. The AST-normalized hashes are all different (because the code structures differ). The execution-result hashes are all identical.

Five code styles, five different AST hashes, one identical execution-result hash. Consensus achieved.

Implementation	AST Hash (first 8 chars)	Exec Hash
Inlined loop	672c33f4	93bcc9f6
Helper function	dbfd4eda	93bcc9f6
List comprehension	9e27d4f9	93bcc9f6
<code>filter</code> +lambda	637dfdf4	93bcc9f6
While loop	c17a323e	93bcc9f6

Heterogeneous Machine Verification

To verify cross-machine reproducibility, the 3M-parameter model was serialized (`async_model.pt`, 3.3 MB) and transferred to a second Linux x86_64 server. Both machines ran the identical inference script with seed 42 and identical temperature.

The hashes matched exactly, confirming that the deterministic inference pipeline is portable across hardware instances of the same class.

Malicious Node Detection

A normal node producing a consensus hash and a malicious node with a deliberately altered parameter (loop bound changed from $\sqrt{n}$ to $\sqrt{n} \div 2$) were simulated. The malicious node’s output hash diverged from the consensus, and the system correctly flagged the deviation.

Applications

Distributed Code Verification

The primary application of ClawNet is autonomous code verification in distributed environments where no trusted third party exists. Example scenarios include:

- **Decentralized CI/CD:** Code-generation tasks are distributed across untrusted worker nodes. Consensus hash is the final artifact.
- **Smart contract auditing:** AI-generated Solidity or Rust code is verified by multiple independent agents.

- **Educational autograders:** Student code submissions are hashed and cross-checked without storing raw solutions.

Video Generation: Temporal Consistency via Perceptual Hash Consensus

The problem.

Current generative video models (Sora, Stable Video Diffusion, Runway Gen-3) operate as stochastic processes. The same text prompt and motion parameters produce radically different outputs across different random seeds: character faces, clothing textures, background elements, and object positions vary unpredictably. This “slot-machine” behavior makes video generation unsuitable for production pipelines requiring consistent character identity and scene layout.

Insight from cel animation.

Traditional cel animation operates on a layered model:

1. A single painted background cel is reused across many frames.
2. Character cels are layered on top, with only the moving joints redrawn per frame.
3. Color and shading are applied as separate passes.

This decomposition produces temporal consistency by construction: the background remains identical; only the intended animated regions change.

Hash-constrained generation.

We propose extending ClawNet’s consensus framework to video generation using **perceptual hash constraints**:

$$\text{Hash}_{\text{perceptual}}(\text{frame}_t^{\text{bg}}) \in \mathcal{H}, \text{Hash}_{\text{perceptual}}(\text{frame}_{t \in \dots + \hat{\Delta} \dots 1}^{\text{bg}})$$

and

$$\hat{f}'' \text{Hash}_{\text{perceptual}}(\text{frame}_t^{\text{fg}}, \hat{\Delta} \dagger \text{frame}_{t \in \dots + \hat{\Delta} \dots 1}^{\text{fg}}) \in \mathcal{H}, \hat{\Delta} \sim \hat{\Delta}, [\hat{f} \pm, \hat{\Delta} \dagger \hat{f}^2]$$

where:

- $\text{frame}_t^{\text{bg}}$ is the background layer at frame t ,
- $\text{frame}_t^{\text{fg}}$ is the foreground (character, object) layer at frame t ,
- $\text{Hash}_{\text{perceptual}}(\hat{\Delta} \leftarrow \dots)$ is a perceptual image hash (e.g., pHash, dHash, or wavelet hash) that is robust to color/lighting variations but sensitive to structural changes,
- $\hat{f}'' \text{Hash}_{\text{perceptual}}$ measures the Hamming distance between adjacent-frame perceptual hashes.

Multi-node video verification.

Under the ClawNet framework, each video generation node produces a per-frame hash vector: $\mathbf{H}_n \in \mathcal{H} \times \mathcal{H} \times \dots \times \mathcal{H}$, $[h_{\text{frame}_1}, \hat{\Delta} \dagger h_{\text{frame}_2}, \hat{\Delta} \dagger \hat{\Delta}', \hat{\Delta} \dagger h_{\text{frame}_T}]$

Consensus across N nodes is reached when: $\text{Hamming}(\mathbf{H}_i, \mathbf{H}_j) \leq \tau \cdot \tilde{I}_{\text{perceptual}}$

where $\tilde{I}_{\text{perceptual}}$ is a small threshold accounting for negligible differences (e.g., compression artifacts, minor color shifts). Nodes whose frame hash vectors deviate beyond $\tilde{I}_{\text{perceptual}}$ are rejected.

Detection of common video artifacts.

This framework can automatically detect:

- **Object flicker:** A foreground object that disappears and reappears causes the frame-by-frame perceptual hash to oscillate between two states, producing a detectable divergence pattern.
- **Face/identity inconsistency:** If a character’s face structure changes between frames, the perceptual hash of the facial region changes, signalling a violation.
- **Background instability:** A steady background should produce identical perceptual hashes across consecutive frames. Any deviation indicates temporal inconsistency.

Implementation path.

A practical implementation would:

1. Use a perceptual hash library (e.g., `ImageHash`) to compute frame hashes at `pHash` granularity.
2. Decompose each frame into background/foreground masks using a pretrained segmentation model (e.g., `SAM`).
3. Compute separate hashes for each layer.
4. Apply consensus rules: background layer must have near-zero hash variance; foreground layer hash must vary smoothly.
5. Redundant generation across $N \geq 3$ nodes provides the same Byzantine fault tolerance as in the code verification use case.

This application transforms the core ClawNet principle—“determinism + hash consensus”—into a practical tool for production video pipelines, where “the same prompt should produce the same video structure” becomes a formal requirement rather than a hope.

Limitations and Future Work

Limitations.

- **Model scale.** Our current validation uses a 3M-parameter model. Scaling to production-grade 185M-parameter inference on CPU-only hardware requires either model quantization or hardware acceleration.
- **Open-ended code generation.** The execution-consensus layer only applies to tasks with deterministic input-output mappings. Creative coding tasks (e.g., “write a blog engine”) cannot be verified via execution.

- **General-purpose LLMs.** The protocol requires a model that produces *exactly* the same output given the same input. Large proprietary models with versioned APIs, unknown internal randomness, or non-deterministic kernel operators cannot participate.

Future work.

- **Video generation validation.** Implement and benchmark the perceptual hash consensus pipeline described in Section 5.2. Key metrics: false-positive rate (background flashing flagged as normal) and false-negative rate (character face change not detected).
- **Cross-language consensus.** Extend execution consensus beyond single-language tasks. A Python implementation and a Go implementation of the same algorithm, when run in containers, should yield identical stdout isomorphisms.
- **Zero-knowledge proof integration.** Replace raw hash disclosure with ZK-SNARK proofs of hash equivalence, so nodes can prove consensus without revealing their generated code.
- **Federated Mother Model training.** Train the Mother Model across multiple untrusted parties using secure aggregation, so no single party controls the model weights.

Conclusion

We presented ClawNet, a distributed code verification protocol that replaces centralized AI judges with deterministic hash consensus. The protocol achieves bit-exact reproducibility across repeated runs, AST-normalized hash matching across naming conventions, and execution- result consensus across syntactically different but semantically equivalent implementations. A secondary experiment confirmed cross-machine portability of the deterministic inference pipeline.

The core contribution is a shift in architectural philosophy: instead of building better judges, we build systems where verification emerges from replicated computation. This principle extends naturally to video generation, where perceptual hash consensus can enforce temporal consistency—a problem that current generative models address only through increasingly large training data and model scale.

All code and experimental data are available at <https://github.com/asteroidea95/ClawNet>.

20

D. Guo et al., “DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code AI,” arXiv:2401.14196, 2024.

Anthropic, “Claude 3.5 Sonnet Model Card,” 2024.

M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.

P. Chen et al., “Codex: Evaluating Large Language Models Trained on Code,” OpenAI, 2021.

L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” arXiv:2306.05685, 2023.

A. Madaan et al., “Self-Refine: Iterative Refinement with Selffeedback,” NeurIPS 2023.

C. Zheng et al., “Position Bias in LLM-based Evaluation,” arXiv:2306.05685, 2023.

L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Trans. Program. Lang. Syst.*, vol. 4, no. 3, pp. 382–401, 1982.

I. D. Baxter et al., "Clone Detection Using Abstract Syntax Trees," *ICSM* 1998.

L. Jiang et al., "DECKARD: Scalable and Accurate Tree-based Detection of Code Clones," *ICSE* 2007.

J. Buchner, "ImageHash: A Python Perceptual Hash Library," <https://github.com/JohannesBuchner/imagehash>, 2013.

A. Kirillov et al., "Segment Anything," *arXiv:2304.02643*, 2023.